

CSC 452/552 Operations Systems

Project 2 The Classic Bounded Buffer Problem

Name: Sean Varie

Bronco ID: 114159195

Date: Oct. 22, 2024

1. Project Overview

Project 2 was to create, using C, a bounded buffer, or more specifically, a thread-safe, blocking queue with a maximum capacity configurable upon creation of the queue. An interface (lab.h) was provided with methods required to be implemented for this queue. A configurable driver program simulating the standard producer-consumer model using the buffer was also provided, as well as a build system (a make file) for building the project and running the provided test code.

2. Project Management Plan

a) Task 1: (Setup)

For task one, I created a branch off of the repository I had previously forked from the provided repository. I then checked out that branch in my codespace.

b) Task 2: (Prepare your repository)

For task two, I exactly followed the instructions, copying the starter code into the appropriate files and committing the changes to my branch.

c) Task 3: (Implement the header file)

Task 3 was to implement the provided lab.h functions. This was the main bulk of the assignment. I chose to implement the buffer as a circular queue, allocating enough data to hold every item when the queue is at capacity. This meant that I also had to create a boolean isEmpty variable in the queue struct to keep track of if the queue was empty or full when both pointers (addIndex and rmIndex in my implementation) are the same. For proper synchronization I chose to have a single mutex, used to ensure that only one queue modifying operation could be performed at a time. This was chosen over having two mutexes, one for adding and one for removing, in order to ensure proper synchronization in particular edge cases, namely when there is only 0 or 1 element in the buffer, and when the buffer is full.

Another potential problem arose which could cause a deadlock if the wrong operation, one which can not occur at the time, locks the mutex. For example, if enqueue locks the mutex while the queue is full, then dequeue cannot lock the mutex in order to make room for the enqueue to occur. Some logic was added to ensure that the mutex will only be locked by an operation when that operation can occur. For efficiency's sake, this logic is set up such that if an operation cannot occur, the thread will wait on a

semaphore, which will be signaled when the state of the queue changes in a way which could allow the operation to occur.

d) Final Task: (Submit your code)

By necessity, at time of writing, this task has not yet been completed. I plan to add and commit this document to my git repository, then download my git repository as a zip and submit via canvas.

3. Project Deliveries

a) How to compile and use

To compile, run the following command from the project directory in github codespaces:

```
$ make
```

To run the producer-consumer driver program, after compiling, run the following command:

```
$ ./myprogram [-c num consumer] [-p num producer] [-i num items] [-s queue size] <-d>
```

The -d flag will introduce a random delay between producer and consumer.

The driver will default to 1 producer, 1 consumer, and 10 items with a queue size of 5 if no arguments are provided, or if only the delay flag is provided:

```
$ ./myprogram
```

```
$ ./myprogram -d
```

To run the test suite, run the following command:

```
$ make check
```

b) Self-modification

The only modifications made to the provided code which are present in the submitted code are the previously described implementation of the queue in the lab.c file.

c) Summary of Results

The queue was tested using the provided driver with many configurations. Up to 200 total threads (minor modification to the driver was required on my local copy) and 100,000 items were tested multiple times. A greater number of consumers than producers, and vice versa, were both tested. Differing queue sizes were also tested. All tests resulted in no loss of elements.

4. Self-Reflection of Project 2

This project was quite simple. The implementation of a circular buffer is very basic and the synchronization of the queue operations is slightly more complex, but not too difficult. There are most likely a number of minor optimizations which could be applied to my code, but nothing which would drastically affect performance when used in a practical system. I believe my experience with multithreading and synchronization within distributed systems helped me significantly in this project.

5. Use of AI for debugging (optional)

No AI was used in the competition of this project.